

Hash Table — What to Learn

1. What is Hashing?

- What is a hash table?
- What is hashing?
- Key-value pairs
- How hashing provides average **$O(1)$** lookup

2. Hash Function

- Converting a key into an index
- Example: `key % table_size`
- Properties of a good hash function

3. Collision

- What is a collision?
- Why collisions happen
- **Chaining**
- **Linear probing**
- **Quadratic probing**
- **Double hashing**

4. Hash Table Operations

- Insert
- Search
- Delete
- Update
- Time complexity
- Average case: **$O(1)$**
- Worst case: **$O(n)$**

5. Hash Table vs Python `dict`

- Hash Table → general data structure/concept
- `dict` → Python's built-in hash-table-based mapping
- Understanding how `dict` works internally

6. Python Hash Table Tools

- `dict`
- `set`
- `.get()`
- `.keys()`
- `.values()`
- `.items()`

7. Common Use Cases

- **Frequency counting** — count how often elements appear
- **Duplicate detection** — find repeated elements
- **Two Sum** — efficiently find pairs with a target sum
- **Caching** — store previously calculated results

- **Fast lookup** — retrieve data using a key
- **Grouping data** — group items based on a common property
- **Database indexing** — speed up data retrieval
- **Removing duplicates** using a **set**
- **Counting characters/words** in strings or documents

8. Practice Workouts

- Frequency Counter
- Two Sum
- First Non-Repeating Character

Learning Flow

Hashing → Hash Function → Hash Table → Collision → Collision Handling → Operations
→ Complexity → Python **dict/set** → Common Use Cases → Practice Problems

Stacks & Queues

A) Stack

Definition

A **Stack** is a linear data structure in which elements are inserted and removed from the **same end**, called the **TOP**.

A Stack follows the **LIFO (Last In, First Out)** principle, meaning the element inserted last is the first element to be removed.

Example:

```

TOP
↓
[30] ← POP
[20]
[10]
```

Here, **30** was inserted last, so it will be removed first.

Stack Operations

- **PUSH** — Adds an element to the top of the stack.
- **POP** — Removes the top element from the stack.
- **PEEK/TOP** — Returns the top element without removing it.
- **DISPLAY** — Displays the elements in the stack.

Important Concepts

- **Stack Overflow** — Trying to add an element when the stack is full.
- **Stack Underflow** — Trying to remove an element when the stack is empty.
- Implementation using **Array**
- Implementation using **Linked List**
- Time complexity of PUSH, POP, and PEEK — **O(1)**

Common Use Cases

- Function calls and recursion
 - Undo/Redo operations
 - Browser history
 - Parentheses matching
 - Expression evaluation
 - Backtracking
 - Depth-First Search (DFS)
-

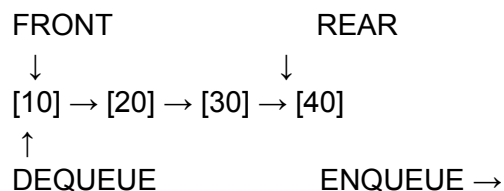
B) Queue

Definition

A **Queue** is a linear data structure in which elements are **inserted at one end (REAR)** and **removed from the other end (FRONT)**.

A Queue follows the **FIFO (First In, First Out)** principle, meaning the element inserted first is the first element to be removed.

Example:



Here, **10** entered first, so it will be removed first.

Queue Operations

- **ENQUEUE** — Adds an element to the rear of the queue.

- **DEQUEUE** — Removes an element from the front of the queue.
- **PEEK/FRONT** — Returns the front element without removing it.
- **DISPLAY** — Displays the elements in the queue.

Important Concepts

- **Queue Overflow** — Trying to add an element when the queue is full.
- **Queue Underflow** — Trying to remove an element when the queue is empty.
- Implementation using **Array**
- Implementation using **Linked List**
- Time complexity of ENQUEUE and DEQUEUE — **O(1)**

Common Use Cases

- CPU/process scheduling
- Printer/task scheduling
- Customer service systems
- Message queues
- Breadth-First Search (BFS)
- Network request handling
- Traffic/line management

Stack vs Queue

Feature	Stack	Queue
Principle	LIFO	FIFO
Full form	Last In, First Out	First In, First Out
Insertion	PUSH	ENQUEUE
Deletion	POP	DEQUEUE
Insertion point	TOP	REAR
Deletion point	TOP	FRONT

Practice

1. Implement a **Stack** with PUSH, POP, PEEK, and DISPLAY.
2. Implement a **Queue** with ENQUEUE, DEQUEUE, PEEK, and DISPLAY.
3. Handle **overflow** and **underflow** conditions.

4. Implement both using an **array**.
5. Implement both using a **linked list**.
6. Solve at least **3 stack/queue problems**.

Trees & Binary Search Trees

1. Tree

A **tree** is a non-linear data structure used to represent hierarchical relationships between elements.

Unlike arrays and linked lists, where elements are arranged sequentially, a tree organizes data using **parent-child relationships**.

Basic Terminology

- **Node** → An element that stores data.
- **Root** → The topmost node of a tree.
- **Parent** → A node that has one or more children.
- **Child** → A node directly connected below a parent.
- **Leaf** → A node with no children.
- **Edge** → A connection between two nodes.
- **Subtree** → A smaller tree formed from a node and its descendants.
- **Depth** → Number of edges from the root to a node.
- **Height** → Number of edges on the longest path from a node to a leaf.

Example:

```
  10
 /  \
5    20
/\
2  7
```

Here, **10** is the root, **5** and **20** are its children, and **2**, **7**, and **20** are leaf nodes.

2. Binary Tree

A **Binary Tree** is a tree in which each node can have **at most two children**.

The two children are called:

- **Left child**
- **Right child**

Example:

```
  10
 /  \
5    20
/\
2  7
```

A node can have:

- 0 children
- 1 child
- 2 children

but never more than two children.

Important Point

A binary tree does **not** require any ordering between values.

For example:

```
  10
 /  \
50   2
```

is still a valid binary tree.

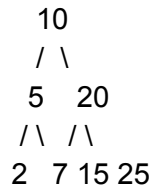
3. Binary Search Tree (BST)

A **Binary Search Tree (BST)** is a special type of binary tree that follows an ordering rule.

For every node:

Left subtree < Node < Right subtree

Example:



For node 10:

Left side → values smaller than 10

Right side → values greater than 10

For node 5:

Left side → values smaller than 5

Right side → values greater than 5

This ordering allows searching to skip unnecessary parts of the tree.

4. Binary Tree vs BST

Binary Tree	Binary Search Tree
At most two children	At most two children
No ordering requirement	Follows an ordering rule
Values can be arranged arbitrarily	Left values are smaller and right values are greater
Searching may require checking many nodes	Searching can skip parts of the tree
General hierarchical structure	Useful for maintaining ordered data

5. BST Operations

Common BST operations include:

1. **Insertion**
 2. **Contains / Search**
 3. **Deletion**
 4. **Tree Traversals**
 5. **Finding the Closest Value**
 6. **Validating a BST**
-

6. BST Insertion

Insertion means adding a new value to a BST while maintaining the BST property.

Process

1. Start at the root.
2. Compare the new value with the current node.
3. If the value is smaller, move to the left subtree.
4. If the value is larger, move to the right subtree.
5. Continue until an empty position is found.
6. Insert the new node there.

Example:

Insert 7:

```
  10
 /
5
 /
2
```

Since:

$7 < 10 \rightarrow$ move left

$7 > 5 \rightarrow$ move right

Result:

```
  10
```

```
  /
 5
 /\
2 7
```

7. Contains / Search

Contains checks whether a particular value exists in the BST.

At every node:

Target == current → Found

Target < current → Move left

Target > current → Move right

Example:

```
  10
  /
  5
  \
  7
```

To search for 7:

10 → 5 → 7

The BST property allows us to eliminate one side of the tree at every step.

8. BST Deletion

Deletion means removing a node while keeping the tree a valid BST.

There are three main cases.

Case 1: Node is a Leaf

A leaf has no children.

```
10
 /
5
```

Deleting 5:

```
10
```

The node can simply be removed.

Case 2: Node Has One Child

Example:

```
10
 /
5
 \
7
```

If 5 is deleted, its child 7 takes its position:

```
10
 /
7
```

Case 3: Node Has Two Children

Example:

```
10
 / \
5  20
```

/ \
15 25

If 20 is deleted, it must be replaced by a suitable value.

Two common choices are:

- **Inorder successor** → smallest value in the right subtree.
- **Inorder predecessor** → largest value in the left subtree.

For 20, the inorder successor is 25.

Result:

10
/ \
5 25
/
15

The BST property is preserved.

9. Tree Traversals

Traversal means visiting every node in a specific order.

The main traversals are:

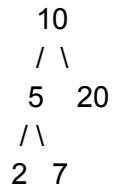
- Inorder
- Preorder
- Postorder
- Level Order

Inorder Traversal

Order:

Left → Root → Right

Example:



Inorder result:

$2 \rightarrow 5 \rightarrow 7 \rightarrow 10 \rightarrow 20$

Important

Inorder traversal of a valid BST produces the values in **sorted order**.

Preorder Traversal

Order:

Root $\rightarrow$ Left $\rightarrow$ Right

Result:

$10 \rightarrow 5 \rightarrow 2 \rightarrow 7 \rightarrow 20$

Postorder Traversal

Order:

Left $\rightarrow$ Right $\rightarrow$ Root

Result:

$2 \rightarrow 7 \rightarrow 5 \rightarrow 20 \rightarrow 10$

Level Order Traversal

Nodes are visited **level by level**.

Result:

$10 \rightarrow 5 \rightarrow 20 \rightarrow 2 \rightarrow 7$

Level order traversal is commonly implemented using a **queue**.

10. Closest Value in a BST

The **closest value problem** asks us to find the value in a BST that has the smallest numerical difference from a given target.

Example:

```
    10
   / \
  5  20
 / \
2  7
```

Target:

8

Compare:

$|8 - 7| = 1$
 $|8 - 10| = 2$

Therefore, **7** is closer.

Main Idea

While traversing the BST:

1. Keep track of the closest value found so far.
 2. Compare the current node with the target.
 3. Update the closest value if the current node is closer.
 4. Use the BST property to decide whether to move left or right.
-

11. Validating a BST

A tree is a valid BST only when **every node follows the BST ordering rule**.

The rule is:

All values in the left subtree $<$ node

All values in the right subtree $>$ node

Consider:

```
  10
 /  \
5    20
 /
7
```

At first, $7 < 20$, so it appears to be a valid left child of 20 .

However, 7 is inside the **right subtree of 10** .

Therefore, 7 must be greater than 10 .

But:

$7 < 10$

So this is **not a valid BST**.

Key Idea

When validating a BST, we must consider the restrictions created by **all ancestors**, not just the immediate parent.

Each node must remain within a valid range of allowed values.

12. BST Time Complexity

For a reasonably balanced BST:

Operation	Average Time
-----------	--------------

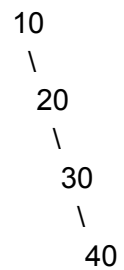
Search	$O(\log n)$
--------	-------------

Insert	$O(\log n)$
--------	-------------

Delete	$O(\log n)$
--------	-------------

However, a BST can become highly unbalanced.

Example:



This starts behaving like a linked list.

In the worst case:

Operation	Worst Case
-----------	------------

Search	$O(n)$
--------	--------

Insert	$O(n)$
--------	--------

Delete	$O(n)$
--------	--------

This is why maintaining tree balance is important in advanced data structures such as **AVL Trees** and **Red-Black Trees**.

Graphs

1. Graph

A **Graph** is a non-linear data structure used to represent relationships or connections between objects.

A graph consists of:

- **Vertices (Nodes)** → Represent objects.
- **Edges** → Represent connections between vertices.

Example:

```
A
 /\
B---C
 \
  D
```

Here:

- A, B, C, D are **vertices**.
- The lines connecting them are **edges**.

Graphs are commonly used to represent:

- Social networks
 - Computer networks
 - Road maps
 - Flight routes
 - Web pages and links
 - Dependencies between tasks
-

2. Types of Graphs

Undirected Graph

In an **undirected graph**, an edge has no specific direction.

$A \text{ ---- } B$

The connection works both ways:

$A \rightarrow B$

$B \rightarrow A$

Example: A friendship relationship.

Directed Graph

In a **directed graph**, every edge has a direction.

$A \text{ ----> } B$

This means:

$A \rightarrow B$

but it does not necessarily mean:

$B \rightarrow A$

Example: Following someone on a social media platform.

Weighted Graph

A **weighted graph** assigns a value or cost to each edge.

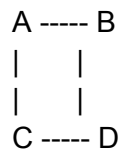
$A \text{ ----}5\text{---- } B$
 $\backslash \quad /$
 2 3
 $\backslash \quad /$
 C

The numbers can represent:

- Distance
 - Cost
 - Time
 - Network bandwidth
 - Travel duration
-

Unweighted Graph

An **unweighted graph** has edges without numerical weights.

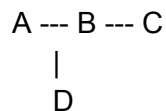


Every connection is treated equally.

3. Connected and Disconnected Graphs

Connected Graph

A graph is **connected** if there is a path between every pair of vertices.



All nodes can be reached from other nodes.

Disconnected Graph

A graph is **disconnected** when some vertices or groups of vertices have no connection.



There is no path from A to C.

4. Graph Representation

A graph can be represented mainly using:

1. **Adjacency Matrix**
2. **Adjacency List**

Adjacency Matrix

An adjacency matrix uses a **2D matrix** to represent connections.

For:

A --- B
|
C

The matrix stores whether an edge exists between each pair of vertices.

For an unweighted graph:

1 → Edge exists
0 → No edge

Advantages

- Easy to check whether two vertices are directly connected.
- Simple representation.

Disadvantages

- Requires $O(V^2)$ space.
 - Can waste space when the graph has relatively few edges.
-

Adjacency List

An adjacency list stores the neighbors of each vertex.

For:

A --- B
|
C

It can be represented as:

$A \rightarrow B, C$
 $B \rightarrow A$
 $C \rightarrow A$

Advantages

- Uses less memory for sparse graphs.
- Efficient for traversing neighboring vertices.

Disadvantages

- Checking whether a specific edge exists can take longer than with a matrix.
-

5. Graph Traversal

Graph traversal means visiting the vertices of a graph systematically.

The two main graph traversal techniques are:

- **BFS — Breadth-First Search**
 - **DFS — Depth-First Search**
-

6. Breadth-First Search (BFS)

BFS explores a graph **level by level**.

It first visits the starting vertex, then all of its immediate neighbors, then the neighbors of those vertices, and so on.

BFS uses a **Queue**.

Basic Idea

Start



Visit immediate neighbors

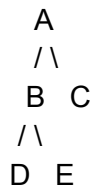


Visit next level



Continue until all reachable nodes are visited

Example:



Starting from **A**:

$A \rightarrow B \rightarrow C \rightarrow D \rightarrow E$

BFS Process

1. Put the starting node into the queue.
2. Mark it as visited.
3. Remove a node from the front of the queue.
4. Visit its unvisited neighbors.
5. Add those neighbors to the queue.
6. Repeat until the queue becomes empty.

Important Data Structure

BFS $\rightarrow$ Queue

7. Depth-First Search (DFS)

DFS explores as far as possible along one path before going back and exploring another path.

DFS can be implemented using:

- **Recursion**
- **Stack**

Example:

```
A
 /\
B  C
 /\
D  E
```

A possible DFS traversal starting from **A** is:

$A \rightarrow B \rightarrow D \rightarrow E \rightarrow C$

Basic Idea

Start

↓

Go as deep as possible

↓

Reach a dead end

↓

Backtrack

↓

Explore another path

Important Data Structure

DFS → Stack / Recursion

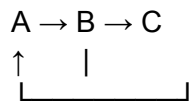
8. BFS vs DFS

BFS	DFS
Breadth-first	Depth-first
Explores level by level	Explores one path deeply
Uses Queue	Uses Stack or Recursion
Useful for shortest path in an unweighted graph	Useful for exploring paths and connectivity
Can require more memory for wide graphs	Can require more memory for deep graphs

9. Visited Set

Graphs can contain **cycles**.

Example:



If we keep following edges without tracking visited nodes, traversal could continue indefinitely.

Therefore, BFS and DFS generally maintain a **visited set**.

The visited set records nodes that have already been processed.

Example:

visited = {A, B, C}

When a node is encountered again, it can be skipped.

10. BFS Applications

BFS is commonly used for:

- Finding the shortest path in an **unweighted graph**
- Finding nodes at a particular distance
- Level-order exploration
- Finding connected components
- Network broadcasting
- Finding relationships within a certain number of connections

Example:

In a social network, BFS can find people who are:

1 connection away
2 connections away
3 connections away

11. DFS Applications

DFS is commonly used for:

- Exploring all reachable nodes
 - Detecting cycles
 - Finding connected components
 - Solving maze/path problems
 - Topological sorting
 - Checking graph connectivity
 - Exploring dependencies
-

12. Graph Traversal Complexity

For both BFS and DFS, when using an adjacency list:

Time Complexity = $O(V + E)$

Where:

- **V** = Number of vertices
- **E** = Number of edges

Why?

Each vertex is visited at most once, and each edge is examined during traversal.

Space Complexity

$O(V)$

because the algorithm may need to store:

- Visited vertices
 - Queue for BFS
 - Stack/recursion for DFS
-

13. Common Graph Traversal Problems

Graph traversal techniques can be used to solve problems such as:

Problem 1: Find Connected Components

Determine how many separate groups exist in a graph.

A --- B C --- D

E

There are two connected components:

Component 1 → A, B

Component 2 → C, D

Component 3 → E

BFS or DFS can be used to discover each component.

Problem 2: Detect a Cycle

Determine whether a graph contains a cycle.

Example:

```
A --- B
|   |
D --- C
```

There is a cycle:

$A \rightarrow B \rightarrow C \rightarrow D \rightarrow A$

DFS is commonly used for cycle detection.

Problem 3: Find the Shortest Path

Given two vertices, find the shortest path between them.

For an **unweighted graph**, BFS can find the shortest path because it explores nodes level by level.

Example:

```
A --- B --- C
|       |
D ----- E
```

Starting from A, BFS explores the graph by distance from A.

14. Key Points to Remember

Graph

- Vertices
- Edges

Graph Representations

- Adjacency Matrix
- Adjacency List

Traversal

- └─ BFS → Queue
- └─ DFS → Stack / Recursion

BFS

- └─ Level by level

DFS

- └─ Go deep → Backtrack

Traversal Complexity

- └─ $O(V + E)$

Applications of Data Structures

The main data structures covered this week are:

1. **Hash Table**
2. **Stack**
3. **Queue**
4. **Tree**
5. **Graph**

Each structure is designed for different types of problems and real-world applications.

1. Hash Table

A **Hash Table** stores data in **key-value pairs** and uses a hash function to determine where the data should be stored.

Example:

Key	Value
name	Rashik
age	18
course	BCA

Main Advantage

Hash tables provide very fast average-case:

Search $\rightarrow O(1)$

Insert $\rightarrow O(1)$

Delete $\rightarrow O(1)$

Applications

1. Dictionaries

Programming languages use hash-table-like structures to implement dictionaries.

Example:

```
student = {  
    "name": "Rashik",  
    "age": 18  
}
```

2. Caching

Frequently used data can be stored in a hash table so it can be retrieved quickly.

Examples:

- Web caches
- API response caches
- Database caches

3. Database Indexing

Hash-based indexes can be used to quickly locate records using a key.

Example:

User ID $\rightarrow$ User information

4. Counting Frequencies

Hash tables are useful for counting how often something occurs.

Example:

apple $\rightarrow 3$

banana → 2
orange → 1

5. Removing Duplicates

A hash set can keep track of values that have already appeared.

6. Symbol Tables

Compilers and interpreters can use hash tables to store information about:

- Variables
 - Functions
 - Identifiers
 - Scope information
-

2. Stack

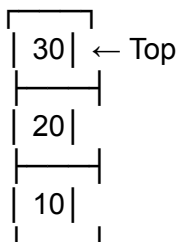
A **Stack** follows the:

LIFO

Last In, First Out

principle.

Example:



The last element inserted is the first one removed.

Main Operations

Push → Add an element

Pop → Remove the top element
Peek → View the top element

Applications

1. Function Calls

Programming languages use a **call stack** to keep track of function calls.

```
main()
  ↓
function A()
  ↓
function B()
```

When **B()** finishes, control returns to **A()**.

2. Undo/Redo

Applications such as text editors use stacks to keep track of previous actions.

Type → Delete → Format

Undo removes the most recent action first.

3. Browser History

A stack-like structure can be used to manage navigation history.

Page A → Page B → Page C

Back navigation moves toward the most recently visited previous page.

4. Parentheses Matching

Stacks are used to check whether brackets are properly balanced.

Example:

{([])}

Opening brackets are pushed onto the stack and matched when closing brackets appear.

5. Expression Evaluation

Stacks are used in:

- Infix expressions
- Prefix expressions
- Postfix expressions

They are also useful for evaluating arithmetic expressions.

6. DFS

Depth-First Search can be implemented using a stack.

DFS → Stack

3. Queue

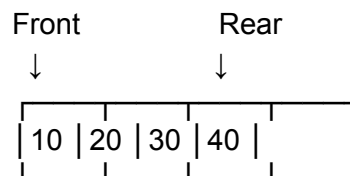
A **Queue** follows the:

FIFO

First In, First Out

principle.

Example:



The first element inserted is the first one removed.

Main Operations

Enqueue → Add an element

Dequeue → Remove the front element

Front → View the first element

Applications

1. Printer Queue

When multiple documents are sent to a printer:

Document A
Document B
Document C

The printer generally processes them in order.

2. CPU Scheduling

Operating systems can use queues to manage processes waiting for CPU time.

3. Customer Service Systems

Customers can be processed in the order they arrive.

Customer A → Customer B → Customer C

4. Network Data

Queues can temporarily store packets waiting to be processed or transmitted.

5. BFS

Breadth-First Search uses a queue.

BFS → Queue

The queue allows BFS to process nodes level by level.

6. Task Scheduling

Tasks waiting to be executed can be placed into a queue.

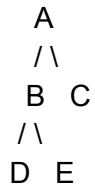
Examples:

- Background jobs
- Download queues
- Print jobs
- Message processing

4. Tree

A **Tree** is a hierarchical data structure consisting of nodes connected by edges.

Example:

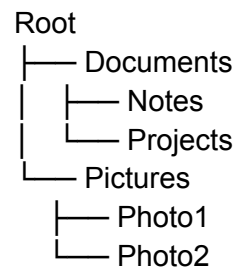


Trees are useful when data naturally has a hierarchical relationship.

Applications

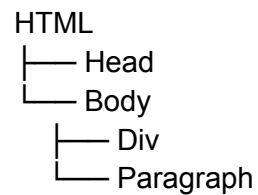
1. File Systems

Operating systems represent folders and files hierarchically.



2. HTML DOM

Web pages are represented as a tree structure.



3. Database Indexing

Structures such as **B-Trees** and **B+ Trees** are widely used in database systems and file systems for efficient searching.

4. Binary Search Trees

A BST maintains ordered data.

It can support:

Search
Insertion
Deletion

efficiently when reasonably balanced.

5. Priority Queues

A **Heap** is a tree-based structure commonly used to implement priority queues.

It allows the highest- or lowest-priority element to be accessed efficiently.

6. Decision Trees

Decision trees are used in **machine learning** for tasks such as:

- Classification
- Prediction

Example:

```
    Age?
   /  \
  /    \
< 18  >= 18
 /      \
Student  Adult
```

7. Autocomplete and Searching

Trie trees can be used for:

- Autocomplete
- Dictionary lookup
- Prefix searching
- Spell checking

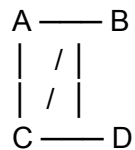
5. Graph

A **Graph** consists of:

- **Vertices (nodes)**
- **Edges (connections)**

Graphs are useful for representing relationships between different entities.

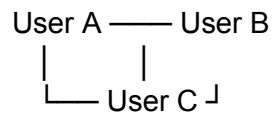
Example:



Applications

1. Social Networks

Users can be represented as vertices and relationships as edges.

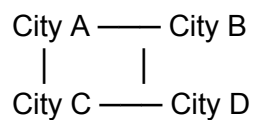


Graphs can represent:

- Friendships
- Followers
- Connections

2. Maps and Navigation

Locations can be represented as vertices and roads as edges.

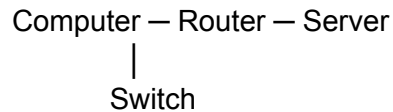


Weights can represent:

- Distance
- Time
- Cost

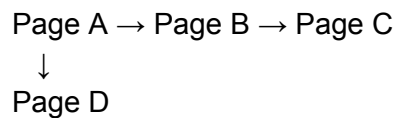
3. Computer Networks

Devices can be represented as nodes and network connections as edges.



4. Web Page Links

Web pages can be represented as nodes and hyperlinks as directed edges.



5. Recommendation Systems

Graphs can represent relationships between:

Users ↔ Products
Users ↔ Movies
Users ↔ Songs

These relationships can help identify related items.

6. Dependency Management

Graphs can represent dependencies between tasks or software packages.

$A \rightarrow B \rightarrow C$

This means **B** depends on **A**, and **C** depends on **B**.

7. BFS and DFS

Graphs are commonly traversed using:

BFS → Queue

DFS → Stack / Recursion

These techniques can be used for:

- Finding reachable nodes
 - Finding paths
 - Detecting cycles
 - Finding connected components
-

6. Choosing the Right Data Structure

Data Structure	Main Idea	Common Applications
Hash Table	Fast key-based lookup	Dictionaries, caching, frequency counting
Stack	LIFO	Undo, function calls, parentheses, DFS
Queue	FIFO	Scheduling, buffering, BFS
Tree	Hierarchical data	File systems, databases, DOM, BST
Graph	Relationships/connections	Maps, social networks, networks, dependencies

7. Quick Memory Trick

Hash Table → "Find quickly"

Stack → "Last comes out first"

Queue → "First comes out first"

Tree → "Hierarchy"

Graph → "Relationships"

The important skill is not just knowing how each structure works, but understanding **which structure naturally fits a particular problem.**